LAPORAN WORKSHOP DEVELOPER OPERATIONAL

MINGGU KE- 9

MATERI PEMBAHASAN

Mengenal dan konfigurasi Docker Swarm

Oleh

Kholifatin Khoiriyah

E41230118

Golongan A

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

2025

DAFTAR ISI

DAFTAR ISI 2
BAB I. HASIL DAN PEMBAHASAN 3
1.1 Acara 17 Mengenal dan konfigurasi Docker Swarm 3
1.2 Acara 18 Mengelola Service pada Docker Swarm 6

BAB I. HASIL DAN PEMBAHASAN

1.1 Acara 17 Mengenal dan konfigurasi Docker Swarm

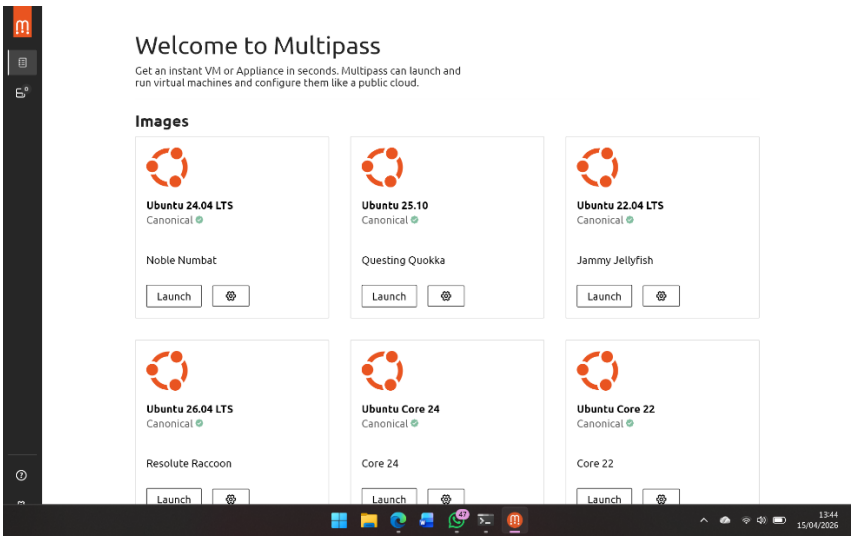
a. Dasar Teori

Docker Swarm merupakan conatainer orchestration tool, yang akan membuat beberapa cluster dari docker host dan merubah mereka menjadi sebuah single virtual server.

b. Prosedur Kerja

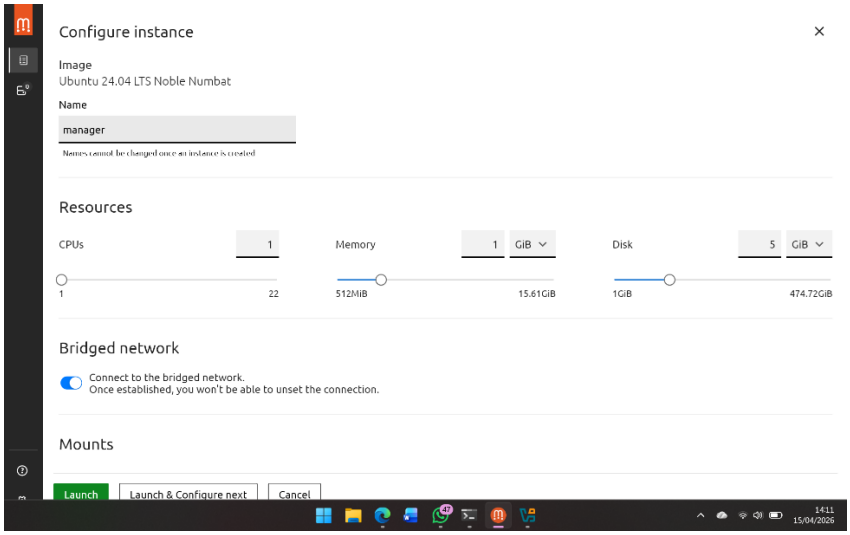
1. Instalasi Multipass

   Instalasi Multipass dilakukan untuk membuat dan mengelola mesin virtual Ubuntu secara ringan dengan konfigurasi yang lebih sederhana.

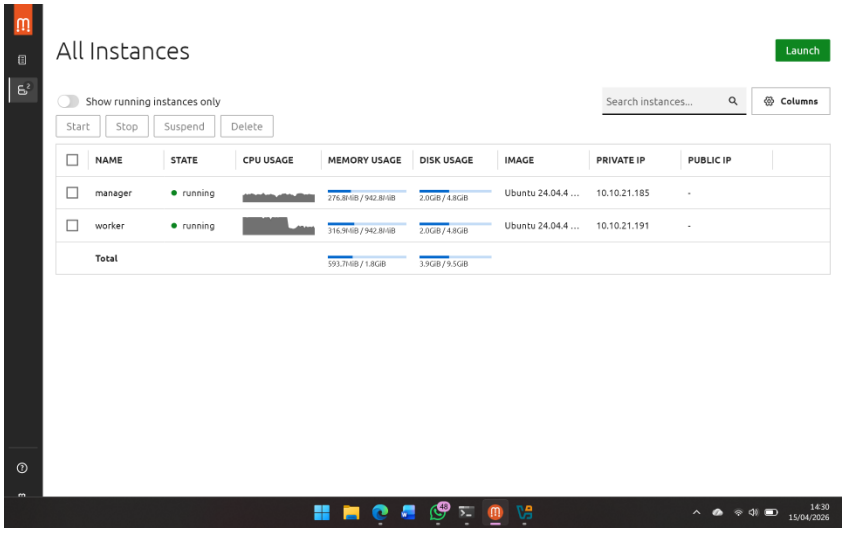


2. Membuat 2 intance pada Multipass dan persiapan Docker Swarm

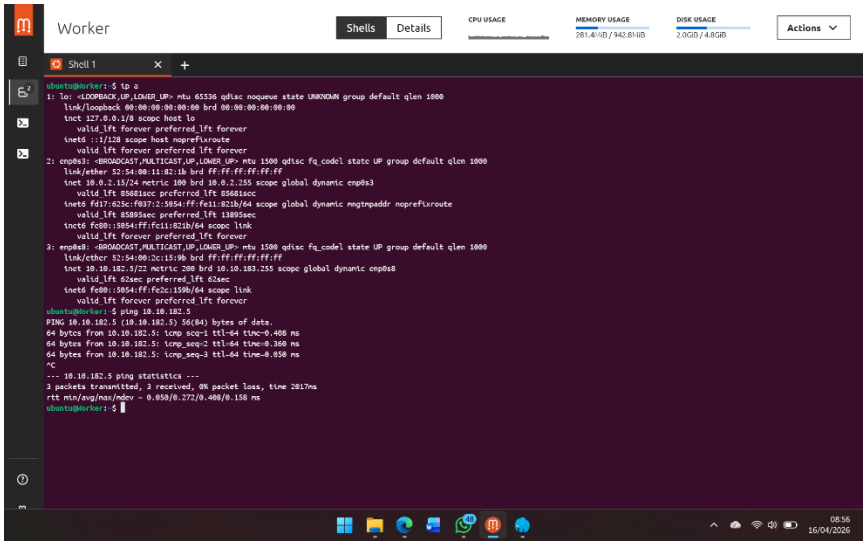
   1) Gunakan ubuntu versi LTS, kemudian lakukan konfigurasi awal instance dengan nama “Manager”, lalu atur spesifikasi perangkat menjadi 1 coreCPU, RAM 1GB, dan penyimpanan 4GB

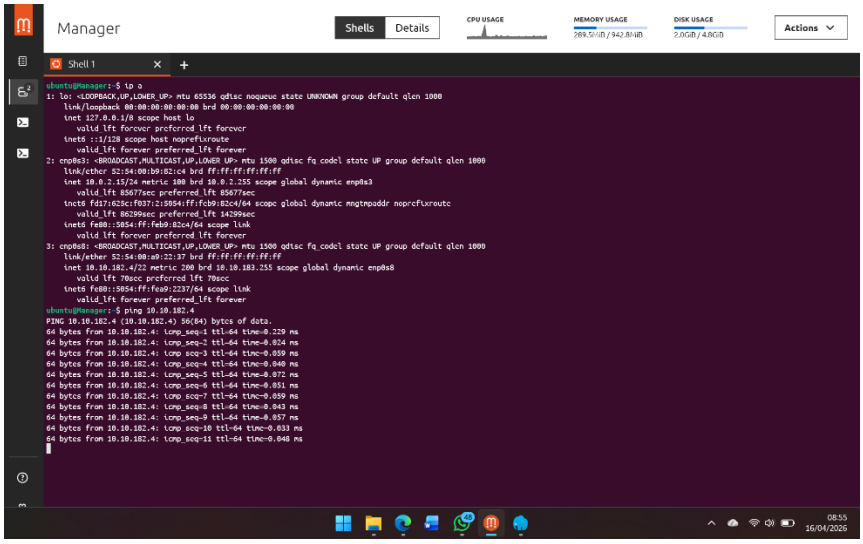


2) Buat satu instance lagi dengan nama yang berbeda, dengan menggunakan spesifikasi yang sama, disini contohnya menggunakan nama "worker"

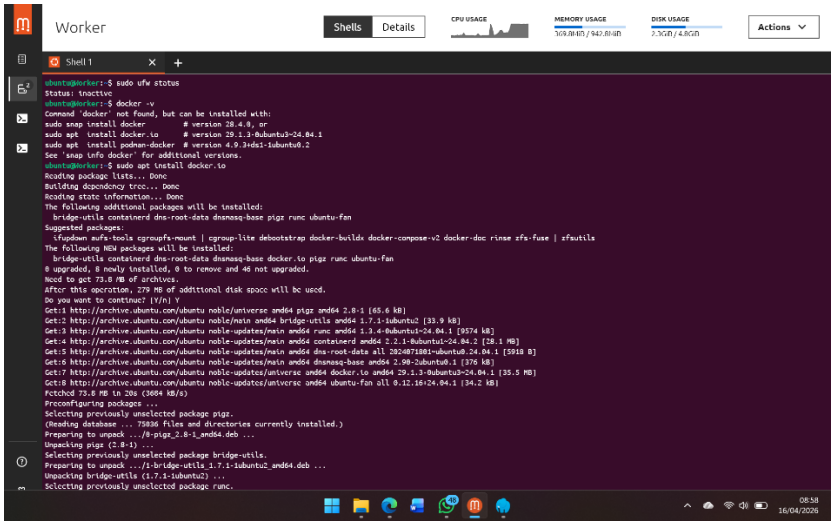


3) Selanjutnya jalankan kedua instance tersebut, dan periksa IP addres dari masing-masing VM dengan menggunakan perintah "ip a". Dan lakukan pengecekan menggunakan perintah "ping" ke alamat IP masing-masing.





4) Selanjutnya, periksa kondisi firewall menggunakan perintah “sudo ufw status” dan pastikan statusnya inactive, kemudian lanjutkan dengan proses instalasi Docker.



3. Melakukan inisiasi Docker Swarm pada Host Manager dan menambhakan Worker pada Docker Swarm.

```
ubuntu@Manager:~$ docker swarm init --advertise-addr 10.10.182.4
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
ubuntu@Manager:~$ sudo docker swarm init --advertise-addr 10.10.182.4
Swarm initialized: current node (q3mad4eop6mbs3nqkerwi9via) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-4sz9bznsjxhj79w0avr7a32nsdxxe24nzl3pa01tdblozl5hk8-chs4d0t3jbnhy8lzmsbffmtzd 10.10.182.4:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
ubuntu@Manager:~$
```

```
Last login: Thu Apr 16 01:51:18 2026 from 10.0.2.2
ubuntu@Worker:~$ sudo docker swarm join --token SWMTKN-1-4sz9bznsjxhj79w0avr7a32nsdxxe24nzl3pa01tdblozl5hk8-chs4d0t3jbnhy8lzmsbffmtzd 10.10.182.4:2377
This node joined a swarm as a worker.
ubuntu@Worker:~$
```

```
ubuntu@Manager:~$ sudo docker info | grep -i swarm
 Swarm: active
ubuntu@Manager:~$ sudo docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
q3mad4eop6mbs3nqkerwi9via *   Manager    Ready     Active         Leader           29.1.3
yjdx5dvuemzmhdp5wy1bw4t9r     Worker     Ready     Active                          29.1.3
ubuntu@Manager:~$
```

1.2 Acara 18 Mengelola Service pada Docker Swarm

a. Dasar Teori

Docker swarm merupakan container orchestration tool, yang akan membuat beberapa cluster dari docker hosy dan merubah mereka menjadi sebuah single virtual server. Sehingga akan meningkatkan performa dan hight availbality pada layanan.

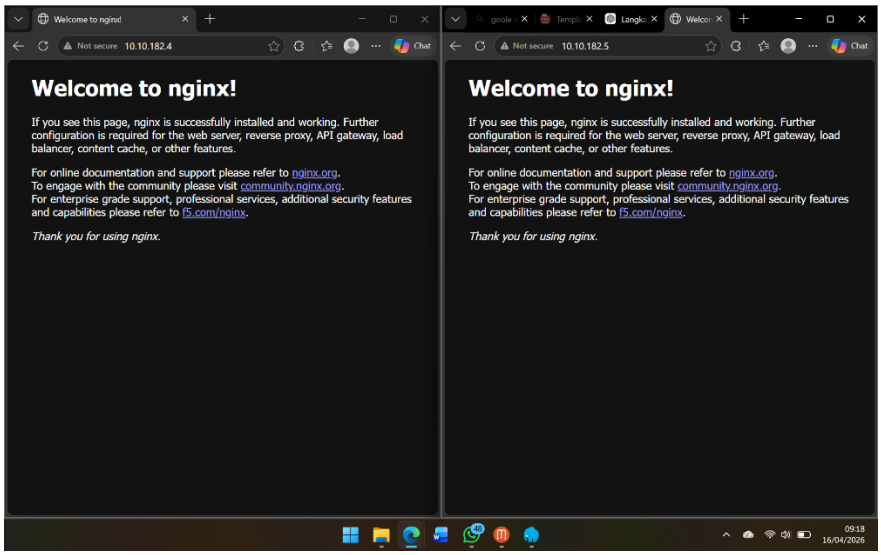
b. Prosedur Kerja

1. Membuat dan Deploy service pada Docker Swarm

1) Service cukup dibuat pada host manager, kemudian akan direplikasi ke host worker menggunakan perintah “docker service crate”. Setelah itu proses pembuatan service. Periksa service yang sudah replikasi dengan perintah docker “service ls”

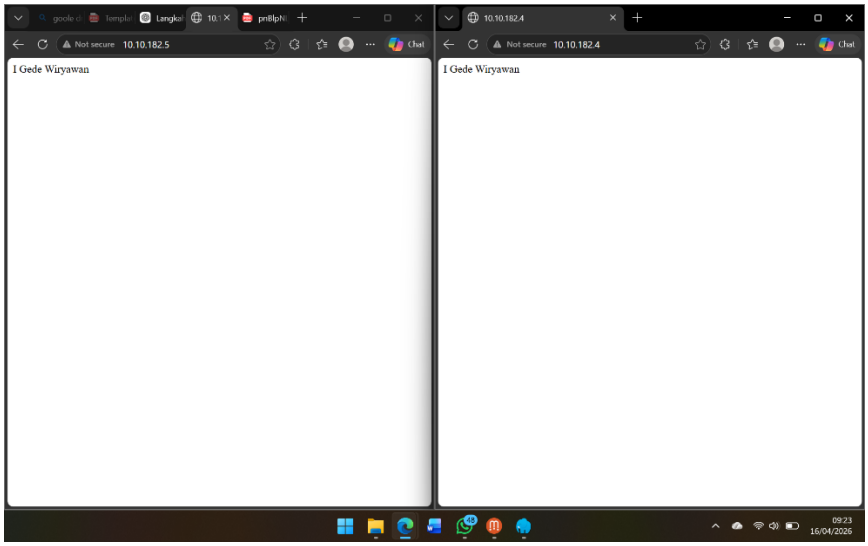
```
ubuntu@Manager:~$ sudo docker info | grep -i swarm
 Swarm: active
ubuntu@Manager:~$ sudo docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
q3mad4eop6mbs3nqkerwi9via *   Manager    Ready     Active         Leader           29.1.3
yjdx5dvuemzmhdp5wy1bw4t9r     Worker     Ready     Active                          29.1.3
ubuntu@Manager:~$ sudo docker service create --name web -p 80:80 nginx
mo1h6h4fezwnnrne4skz4vhbn
overall progress: 1 out of 1 tasks
1/1: running   [==================================================>]
verify: Service mo1h6h4fezwnnrne4skz4vhbn converged
ubuntu@Manager:~$ sudo docker service ls
ID             NAME      MODE         REPLICAS   IMAGE          PORTS
mo1h6h4fezwn   web       replicated   1/1        nginx:latest   *:80->80/tcp
ubuntu@Manager:~$ sudo docker service ps web
ID             NAME      IMAGE          NODE      DESIRED STATE   CURRENT STATE               ERROR     PORTS
jlx1kdeznwtb   web.1     nginx:latest   Manager   Running         Running about a minute ago
ubuntu@Manager:~$
```

2) Selanjutnya akses service tersebut di web browser dengan menggunakan ip address host manager dan worker.



3) Update service dari awalnya menggunakan image nginx menjadi image lain, digunakan image igdw/first-repo yang ada di Docker Hub, dengan perintah “docker service update --image igdw/first-repo web.”

```
ubuntu@Manager:~$ sudo docker service update --image igdw/first-repo web
web
overall progress: 1 out of 1 tasks
1/1: running   [==================================================>]
verify: Service web converged
ubuntu@Manager:~$
```



4) Selanjutnya untuk menginspect service yang sedang berjalan pada host dapat menggunakan perintah “sudo docker service inspect web –pretty”

```
ubuntu@Manager:~$ sudo docker service inspect web --pretty

ID:             mo1h6h4fezwnnrne4skz4vhbn
Name:           web
Service Mode:   Replicated
 Replicas:      1
Placement:
UpdateConfig:
 Parallelism:   1
 On failure:    pause
 Monitoring Period: 5s
 Max failure ratio: 0
 Update order:      stop-first
RollbackConfig:
 Parallelism:   1
 On failure:    pause
 Monitoring Period: 5s
 Max failure ratio: 0
 Rollback order:    stop-first
ContainerSpec:
 Image:         nginx:latest@sha256:7f0adca1fc6c29c8dc49a2e90037a10ba20dc266baaed0988e9fb4d0d8b85ba0
 Init:          false
Resources:
Endpoint Mode:  vip
Ports:
 PublishedPort = 80
  Protocol = tcp
  TargetPort = 80
  PublishMode = ingress
```

5) Host meninggalkan docker swarm

```
ubuntu@Worker:~$ sudo docker swarm leave
Node left the swarm.
ubuntu@Worker:~$
```